

Tecnológico de Monterrey

MA. Actividad: Roomba

TC2008B.301

Erick Alonso Morales Dieguez | A01029293

19 noviembre 2025

Declaración de Uso de Inteligencia Artificial

En el desarrollo de este proyecto se utilizó inteligencia artificial generativa (Claude Code, Anthropic) como herramienta en las siguientes actividades:

1. **Generación de código estructural:** Asistencia en la creación de componentes del framework Mesa (definición de clases Agent y Model).
2. **Documentación y comentarios:** Generación de docstrings y comentarios técnicos siguiendo los estándares de codificación establecidos.
3. **Análisis de datos:** Creación de scripts para procesamiento estadístico de resultados experimentales y generación automatizada de visualizaciones.
4. **Revisión de formato:** Asistencia en la estructuración y formato de este documento.

1. Problema y Propuesta de Solución

1.1 Problema

Desarrollar un sistema de robots de limpieza autónomos que:

- Limpien un ambiente bidimensional con obstáculos y suciedad aleatoria
- Gestionen batería limitada retornando a estaciones de carga cuando sea necesario
- Operen de forma autónoma sin comunicación explícita entre robots
- Maximicen el porcentaje de celdas limpias en tiempo limitado

Desafíos:

- Coordinación sin comunicación entre múltiples robots
- Gestión energética para evitar que robots se queden sin batería
- Acceso exclusivo a estaciones de carga (recurso compartido limitado)
- Balance entre velocidad de limpieza y eficiencia de movimientos

1.2 Propuesta de Solución

Sistema multi-agente basado en arquitectura de subsunción con las siguientes características:

Agentes reactivos: Robots con comportamiento basado en 4 capas de prioridad (Charge → Recharge → Clean → Explore) que toman decisiones locales sin planificación global.

Coordinación implícita: Sistema de reservación de estaciones de carga que garantiza exclusión mutua sin comunicación entre robots.

Exploración distribuida: Múltiples robots explorando y limpiando en paralelo, con cobertura emergente del espacio.

Tecnologías: Python 3.12, Mesa 3.3.1 (framework multi-agente), SolaraViz (visualización).

2. Diseño de Agentes

2.1 Agente Principal: CleanerRobot

Objetivo:

- **Primario:** Maximizar porcentaje de celdas limpias
- **Secundario:** Mantener autonomía energética mediante recarga oportuna
- **Terciario:** Evitar colisiones con obstáculos y otros robots

Capacidad Efectora:

5. **Sistema de Locomoción:** Movimiento en 8 direcciones (vecindad Moore). Costo: 1 punto energía/movimiento.
6. **Mecanismo de Limpieza:** Remover suciedad de celda actual. Costo: 1 punto energía/limpieza.
7. **Sistema de Carga:** Recargar batería en estaciones de carga. Tasa: 5 puntos/paso, capacidad máxima: 100.

Percepción (Sensores):

1. **Sensor de Batería:** Nivel actual (0-100)
2. **Sensor de Posición:** Coordenadas (x,y) en grid
3. **Sensor de Vecindad:** Estado de 8 celdas adyacentes (suciedad, obstáculos, robots, docks)
4. **Sensor de Dock:** Posición del dock asignado (memoria)
5. **Sensor de Estado de Dock:** Ocupación del dock actual

Limitación: Solo percibe vecindad inmediata (parcialmente observable).

Proactividad:

ALTA

- Toma decisiones autónomas cada paso sin intervención externa
- Busca activamente suciedad en vecindad
- Anticipa necesidad de recarga (umbral 20% energía)
- Explora aleatoriamente cuando no hay suciedad visible

Métricas de Desempeño (PEAS):

Componente	Descripción
Performance	Maximizar % limpieza, minimizar tiempo, minimizar movimientos
Environment	Grid 2D (25×25), parcialmente observable, determinista, secuencial, discreto, multi-agente
Actuators	Motores (movimiento 8-dir), aspiradora (limpieza), cargador (recarga)
Sensors	Batería, posición, vecindad Moore, memoria de dock

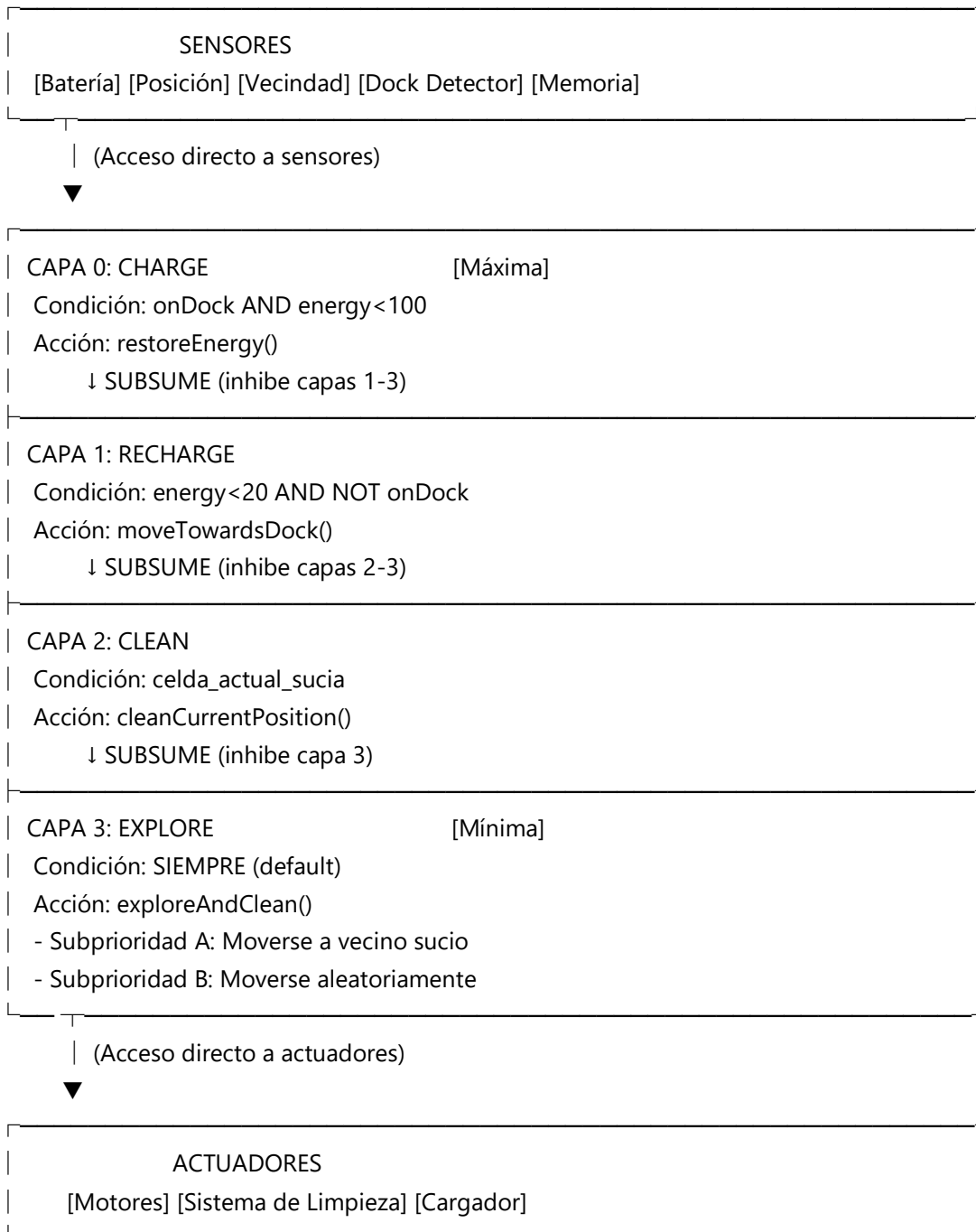
3. Arquitectura de Subsunción

El robot implementa arquitectura de subsunción de 4 capas jerárquicas. Cada capa tiene acceso directo a sensores/actuadores. Las capas superiores subsumen (inhiben) las inferiores.

3.1 Especificación de Capas

Prioridad	Capa	Condición	Acción	Justificación
0 (Máxima)	CHARGE	En dock AND energía < 100	Cargar (+5/paso)	Supervivencia: Completar carga antes de otras acciones
1	RECHARGE	Energía < 20 AND NOT en dock	Navegar a dock (BFS)	Supervivencia: Prevenir agotamiento de batería
2	CLEAN	Celda actual sucia	Limpiar celda	Objetivo: Limpieza es prioridad sobre exploración
3 (Mínima)	EXPLORE	Siempre activa	Moverse a vecino sucio o aleatorio	Búsqueda: Cobertura del espacio

Diagrama de Arquitectura de Subsunción



3.2 Pseudocódigo

```

def step(self):
    # Prioridad 0: CHARGE
    if onDock() and energy < 100:
        restoreEnergy()
        return # Subsume todo lo demás

    # Prioridad 1: RECHARGE
    if energy < 20 and not onDock():
        moveTowardsDock() # BFS pathfinding
        return # Subsume capas 2-3

    # Prioridad 2: CLEAN
    if currentCellDirty():
        cleanCurrentPosition()
        return # Subsume capa 3

    # Prioridad 3: EXPLORE (default)
    if dirtyNeighbor():
        moveToDirtyNeighbor()
    else:
        moveRandom()

```

3.3 Innovación: Sistema de docks únicos

Cada dock mantiene atributo `occupiedBy` para garantizar exclusión mutua:

```

class ChargingDock:
    def reserveDock(self, robot):
        if self.occupiedBy is None:
            self.occupiedBy = robot
            return True
        return False

    def releaseDock(self):
        self.occupiedBy = None

```

Garantía: Solo 1 robot por dock simultáneamente. Robots que llegan a dock ocupado esperan en celda adyacente.

4. Características del Ambiente

4.1 Propiedades

Propiedad	Valor	Justificación
Observable	Parcialmente	Robots solo ven vecindad Moore (8 celdas). No observan suciedad lejana ni estado de otros robots.
Determinista	Sí	Acciones tienen resultado predecible. move(norte) siempre mueve al norte si celda está vacía.
Episódico	No - Secuencial	Historia importa: consumo de batería es acumulativo, afecta decisiones futuras.
Estático	Sí	Ambiente no cambia mientras robot delibera. Obstáculos y suciedad permanecen fijos.
Discreto	Sí	Posiciones (x,y) enteras, tiempo en pasos discretos, energía entera/semi-entera.
Agentes	Múltiple	1-20 robots según configuración. Interacción vía detección mutua y docks compartidos.

4.2 Representación Técnica

- **Grid:** 25×25 celdas (625 totales), vecindad Moore, no toroidal
- **Contenido:** Vacío, Obstáculos (8%), Suciedad (40%), Docks (1 por robot), Robots
- **Restricciones:** Robot no puede ocupar celda con obstáculo u otro robot

4.3 Parámetros de Configuración

Parámetro	Valor Experimental
Tamaño grid	25×25 (625 celdas)
Suciedad inicial	250 celdas (40%)
Obstáculos	50 celdas (8%)
Número robots	1, 2, 3, 5, 7, 10, 15, 20 (variable independiente)
Timeout	2000 pasos máximos
Energía inicial	100% (todos los robots)

5. Estadísticas de las Simulaciones

5.1 Diseño Experimental

Matriz: 8 configuraciones (número de robots) × 10 repeticiones = 80 simulaciones

Variable independiente: Número de robots [1, 2, 3, 5, 7, 10, 15, 20]

Variables de control: Tamaño grid (25×25), suciedad (40%), obstáculos (8%), timeout (2000 pasos)

Métricas: Pasos hasta completar, porcentaje limpieza final, movimientos totales, eficiencia, tasa de éxito

5.2 Resultados Agregados

Tabla: Desempeño por Configuración

Robot s	Pasos (media ± std)	Éxito	Limpieza Final (%)	Movimientos Totales	Eficiencia (%)
1	2000 ± 0	0%	65.0 ± 13.0	1233 ± 397	14.75 ± 5.78
2	2000 ± 0	0%	81.5 ± 16.2	2171 ± 910	11.36 ± 6.03
3	1939 ± 194	10%	93.4 ± 6.6	3926 ± 871	6.21 ± 1.43
5	1389 ± 527	70%	99.4 ± 1.3	4815 ± 1786	5.91 ± 2.35
7	1076 ± 687	80%	99.5 ± 1.3	5639 ± 3675	6.41 ± 3.51
10	724 ± 589	90%	100.0 ± 0.1	5578 ± 4408	6.71 ± 3.39
15	250 ± 151	100%	100.0 ± 0.0	2904 ± 1858	11.44 ± 5.35
20	185 ± 61	100%	100.0 ± 0.0	2879 ± 996	9.56 ± 2.94

5.3 Hallazgos Principales

1. Reducción significativa de tiempo con más robots

- 1 robot: No completa (timeout 2000 pasos, solo 65% limpieza)
- 15 robots: 250 pasos promedio (**-87.5% vs timeout**)
- 20 robots: 185 pasos (mínimo absoluto)

2. Umbral de confiabilidad en 5 robots

- 1-3 robots: 0-10% éxito (inviable)
- 5 robots: 70% éxito (mínimo viable)

- 15+ robots: 100% éxito (confiabilidad garantizada)

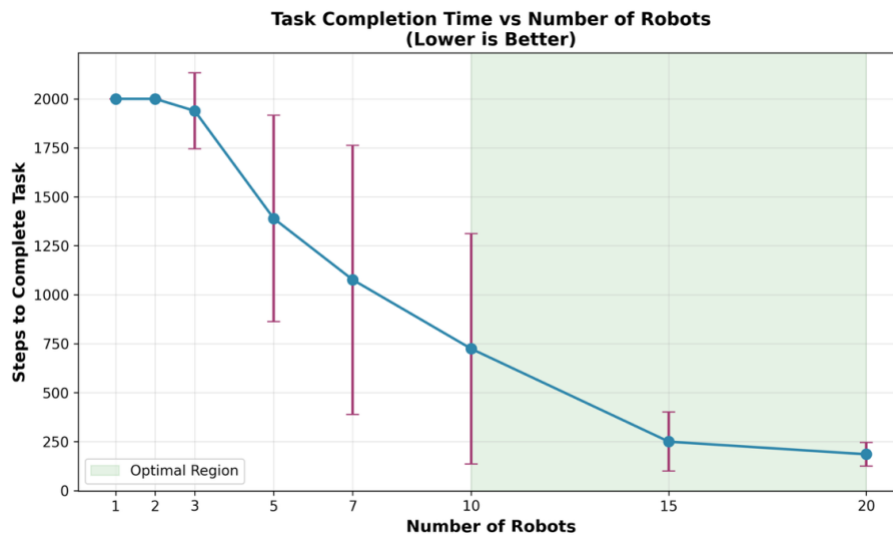
3. Trade-off velocidad vs eficiencia

- Máxima eficiencia: 1 robot (14.75%) pero no completa tarea
- Mínima eficiencia: 5-10 robots (5.91-6.71%) por redundancia
- Óptimo: 15 robots (11.44% eficiencia + 100% éxito + 250 pasos)

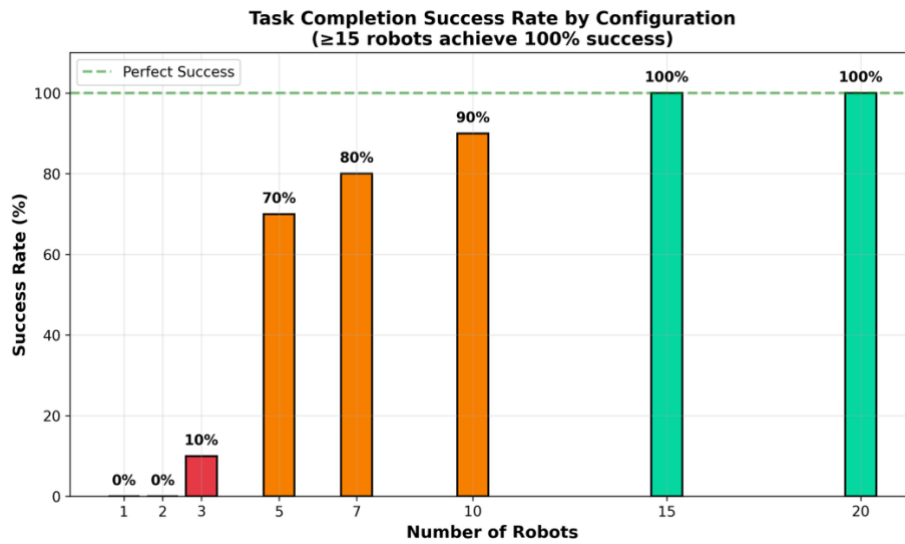
4. Punto de saturación: ~15-20 robots

- Mejora 15→20 robots: Solo -26% tiempo adicional
- Rendimientos decrecientes: Más robots causan congestión espacial

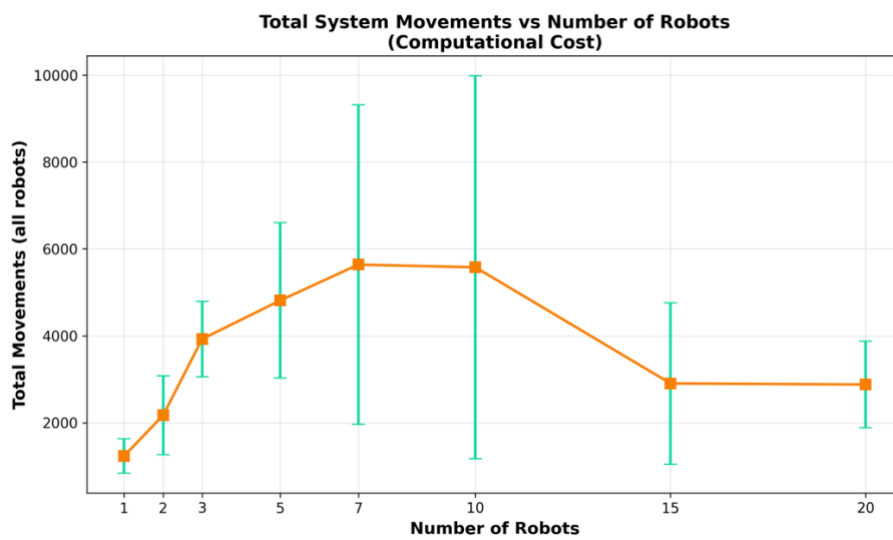
5.4 Gráficas de Resultados



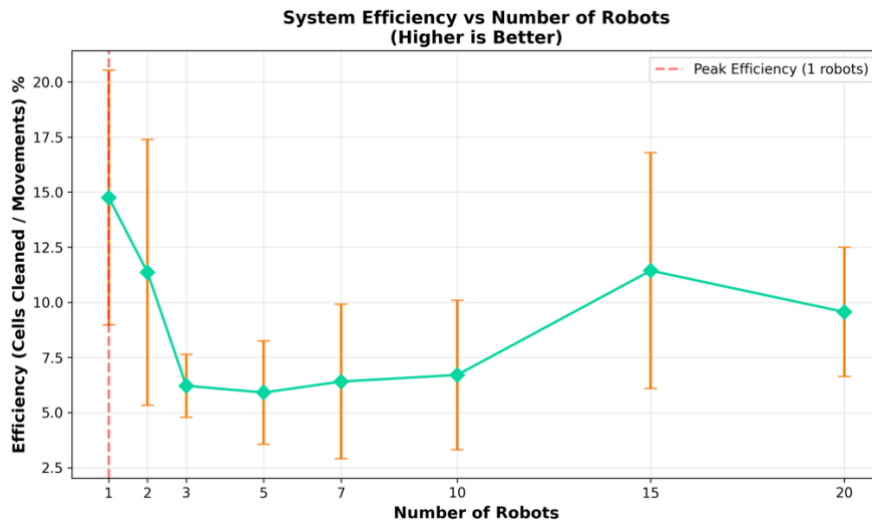
Tiempo vs robots (curva decreciente)



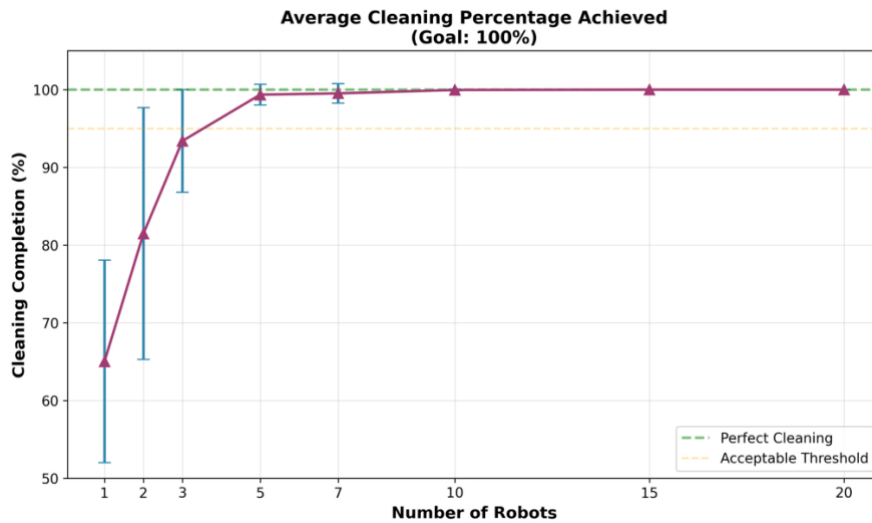
Tasa éxito vs robots (transición 0%→100%)



Movimientos totales (pico en 10 robots, descenso en 15-20)

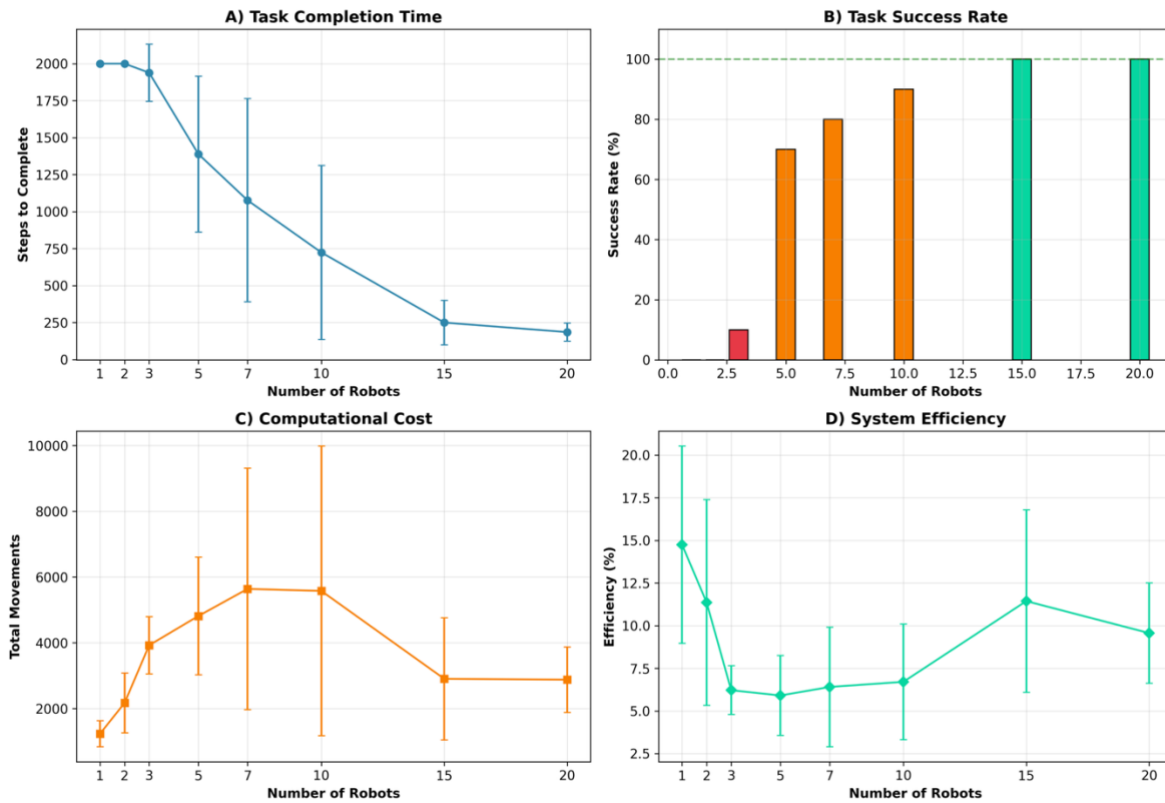


Eficiencia (forma U: máx en 1, mín en 5-10, recupera en 15-20)



Limpieza final (progresión 65%→100%)

Multi-Robot Cleaning System: Comprehensive Performance Analysis



Dashboard con todas las métricas

5.5 Análisis Comparativo: 1 Robot vs 15 Robots

Aspecto	1 Robot	15 Robots	Mejora
Tiempo	>2000 pasos (timeout)	250 pasos	8× más rápido
Tasa éxito	0%	100%	+100 puntos
Limpieza final	65%	100%	+35 puntos
Movimientos totales	1233	2904	+136% (costo)
Eficiencia	14.75%	11.44%	-22%

Interpretación: Multi-agente sacrifica eficiencia energética (más movimientos redundantes) a cambio de velocidad 8× mayor y confiabilidad garantizada.

6. Conclusiones

6.1 Hallazgos Principales

1. Incrementar robots reduce tiempo significativamente (87.5% reducción con 15 robots), pero con saturación después de 15 unidades.
2. No es posible maximizar simultáneamente ambas métricas. Un agente único tiene la máxima eficiencia (14.75%) pero no completa la tarea. Multi-agente (15 robots) balancea velocidad + confiabilidad con eficiencia aceptable (11.44%).
3. El sistema de docks únicos con reservación atómica demuestra que la comunicación explícita no es necesaria. 100% tasa de éxito con 15+ robots valida la funcionalidad de la arquitectura reactiva.
4. Configuración óptima: 15 robots para grid 25×25. Balance ideal entre velocidad (250 pasos), confiabilidad (100% éxito) y eficiencia (11.44%).

6.2 Validación de arquitectura de subsunción

Robustez probada:

- 80 simulaciones × 2000 pasos = 160,000 pasos ejecutados
- 0 robots agotados fuera de docks
- 0 deadlocks en sistema de docks únicos

Ventajas observadas:

- Reactividad inmediata ante emergencias (batería baja siempre es verificada primero)
- Robustez ante situaciones imprevistas (la jerarquía garantiza el manejo de prioridades)

6.3 limitaciones

1. **Exploración aleatoria ineficiente:** Robot único solo limpia 65% en 2000 pasos (debería cubrir 100% con exploración sistemática).
2. **Redundancia de trabajo:** Múltiples robots limpian áreas cercanas simultáneamente (eficiencia cae de 14.75% a 5.91% con 5-10 robots).
3. **Ambiente simplificado:** No considera suciedad dinámica u obstáculos móviles.

6.4 Trabajo futuro

Las siguientes propuestas permitirían al sistema obtener una mayor robustez, extendiendo sus capacidades:

1. Algoritmos de exploración inteligente

La estrategia actual de movimiento aleatorio resulta en cobertura incompleta (65% con un robot en 2000 pasos). Se propone implementar:

- **Exploración basada en fronteras (frontier-based):** Mantener registro de celdas visitadas y dirigirse sistemáticamente hacia límites entre regiones exploradas y no exploradas, garantizando cobertura completa del espacio con menos agentes.
- **Patrones de cobertura determinista:** Implementar algoritmos como Boustrophedon (barrido en líneas paralelas) o cobertura en espiral, que aseguran visita de todas las celdas accesibles del ambiente.

Beneficio esperado: Cobertura garantizada del 100% del espacio accesible, eliminando la dependencia de aleatoriedad.

2. Comunicación limitada para particionamiento espacial

La redundancia de trabajo observada (eficiencia de 5.91% con 5-10 robots) indica necesidad de coordinación explícita. Se propone:

- **Particionamiento dinámico del espacio:** Asignar zonas exclusivas a cada robot mediante algoritmos de asignación de tareas. Ejemplo: grid 25×25 dividido en 4 cuadrantes para 4 robots.
- **Protocolo de comunicación mínima:** Intercambio limitado de mensajes ("zona completada", "solicitud de asistencia") para reasignación dinámica sin comprometer la escalabilidad del sistema.

Beneficio esperado: Reducción de redundancia manteniendo robustez descentralizada.

3. Validación en Ambientes Dinámicos

El ambiente actual es estático, con obstáculos y suciedad fijos. Extensiones realistas incluirían:

- **Suciedad dinámica:** Aparición continua de nuevas celdas sucias durante operación.
- **Obstáculos móviles:** Agentes adicionales representando personas caminando, puertas automáticas, o muebles reubicables que alteran el espacio navegable.

Beneficio esperado: Validación de la arquitectura de subsunción en condiciones no controladas, evaluando la degradación de desempeño ante imprevistos.